



CHAPTER – 6

Data Abstraction and Object Orientation

Object-Oriented Programming

- Control or process abstraction (subroutines!) is a very old idea
- Data abstraction (Object-Oriented (OO)) is somewhat newer
- An Abstract Data Type is one that is defined in terms of the operations that it supports (i.e., that can be performed upon it) rather than in terms of its structure or implementation

Why abstractions?

- ☐ easier to think about - hide what doesn't matter
- ☐ protection - prevent access to things you shouldn't see
- ☐ plug compatibility
 - ✓ replacement of pieces, often without recompilation, definitely without rewriting libraries
 - ✓ division of labor in software projects

We talked about data abstraction some back in the unit on naming and scoping

➤ Historical development of abstraction mechanisms:

❑ Static set of variables	Basic
❑ Locals	Fortran
❑ Statics	Fortran, Algol 60, C
❑ Modules	Modula-2, Ada 83
❑ Module types	Euclid
❑ Objects	Smalltalk, C++, Eiffel, Java ,Oberon, Modula-3, Ada 95

- Statics allow a subroutine to retain values from one invocation to the next, while hiding the name in-between
- Modules allow a **collection of subroutines** to share some statics, still with hiding
- Module types allow the module to be the abstract data type
 - It avoids explicit object parameters to many operations

- The 3 key factors in OO programming
 - ❑ Encapsulation (data hiding)
 - ❑ Inheritance
 - ❑ Dynamic method binding
- OOP is currently ascendant. It cross-cuts paradigms:
 - ❑ Imperative OO (C++, Java, C#, Python, etc.)
 - ❑ Functional OO (Ocaml, CLOS, etc.)
 - ❑ Logical OO (Flora-2)
- The language needs a way of defining a class:
 - ❑ Name,
 - ❑ Super classes (incl. Interfaces),
 - ❑ Fields,
 - ❑ Methods.
 - ❑ Fields + Methods = Members of the class

Benefits of OOP:

1. Reduced Conceptual Load : Give a concept a name ,
 - ❑ To minimizing the amount of detail that the programmer must think about at one time
2. Fault containment: Object can enforce an interface that must be followed
 - ❑ preventing the programmer from using a program component in inappropriate ways, and
 - ❑ limiting the portion of a programs text in which a given component can be used
3. Independence : Reusability
 - ❑ modify their internal implementations without changing external code

Protection

- OO supports data hiding / protection: Keep implementation details from leaking into the larger program
- The 4 kinds of **Visibility** protection in Java:

Encapsulation and Inheritance

- C++ distinguishes among:
 - ❑ public class members : accessible to anybody
 - ❑ protected class members: accessible to members of this or derived classes
 - ❑ Private: accessible just to members of this class
- A C++ structure (struct) is simply a class whose members are public by default
- C++ base classes can also be public, private, or protected

Classes (C++) Example:

class circle : public shape { ...

anybody can convert (assign) a circle* into a shape*

class circle : protected shape { ...

only members and friends of circle or its derived classes can convert (assign) a circle* into a shape*

class circle : private shape { ...

only members and friends of circle can convert (assign) a circle* into a shape*

Initialization and Finalization

Constructor:

- ❑ Most object-oriented languages provide some sort of special mechanism *to initialize an object automatically* at the beginning of its lifetime.
- ❑ A constructor does not allocate space
- ❑ Languages often support multiple constructors.

Destructors:

- ❑ Called when an object goes away, to free up resources used by it.
- ❑ A few languages provide a similar *destructor* mechanism to *finalize an object automatically* at the end of its lifetime

Several important issues arise:

- ❑ choosing a constructor

- o An object-oriented language may permit a class to have zero, one, or many distinct constructors.
- o It may be necessary to distinguish among them by number and types of arguments.

- ❑ references and values

- o If variables are references, then every object must be created explicitly - appropriate constructor is called
- o If variables are values, then object creation can happen implicitly as a result of elaboration

- ❑ execution order

- o When an object of a derived class is created in C++, the constructors for any base classes will be executed before the constructor for the derived class

- ❑ garbage collection

- o If the language implementation collects garbage automatically, then the need for destructors is greatly reduced.

Dynamic Method Binding

- Virtual functions in C++ are an example of **dynamic method binding**
 - you don't know at compile time what type the object referred to by a variable will be at run time

For Example:

- A derived class D has all the members data and subroutines of its base class C.
 - As long as D does not hide any of the publicly visible members of C , it makes sense to allow an object of class D to be used in any context that expects an object of class C:
 - Anything we might want to do to an object of class C we can also do to an object of class D
- Simula also had virtual functions (all of which are abstract)
- In Smalltalk, Eiffel, Modula-3, and Java **all** member functions are virtual

- The ability to use a derived class in a context that expects its base class is called *subtype polymorphism*.

E.g.

```
class person { ...
```

```
    class student : public person { ...
```

```
    class professor : public person { ...
```

```
    student s;
```

```
    professor p;
```

```
    ...
```

```
    person *x = &s;
```

```
    person *y = &p;
```

//Data members of classes are implemented just like structures (records)

//Because both student and professor objects have all the properties of a person object, we should be able to use them in a person context:

//A pointer to the parent and a pointer to the child contain the same address

Virtual and Nonvirtual Methods

- **Non-virtual functions** require no space at run time; the compiler just calls the appropriate version, based on type of variable
- **Virtual functions**
 - Calls to virtual methods are *dispatched to the appropriate* implementation at run time
 - In Simula, C++ and C#, the keyword virtual prefixes the subroutine declaration
 - b/c ,which use **static method binding by default**, the programmer can specify that particular methods should use dynamic binding by labeling them as virtual

Virtual methods in C++

```
class person {
public:
    virtual void print_mailing_label();
    ...
}
```

Virtual methods in Simula

```
CLASS Person;
    VIRTUAL: PROCEDURE PrintMailingLabel;
BEGIN
    ...
    PROCEDURE PrintMailingLabel...
        COMMENT body of subroutine
    ...
END Person;
```

- In most object-oriented languages it is possible to omit the body of a virtual method in a base class.
 - Pure virtual = No implementation for method
- In Java and C#, one does so by labeling both the class and the missing method as abstract:

E.g.

```
abstract class person {  
    ...  
    public abstract void print_mailing_label();  
    ...  
}
```

- Dynamic method binding:
 - Virtual Dispatch
 - The subclass includes methods in the same order = records in the vtable (dispatch table)
 - Multiple inheritance problem.

```

class foo {
    int a;
    double b;
    char c;
public:
    virtual void k( ...
    virtual int l( ...
    virtual void m();
    virtual double n( ...
    ...
} F;

```

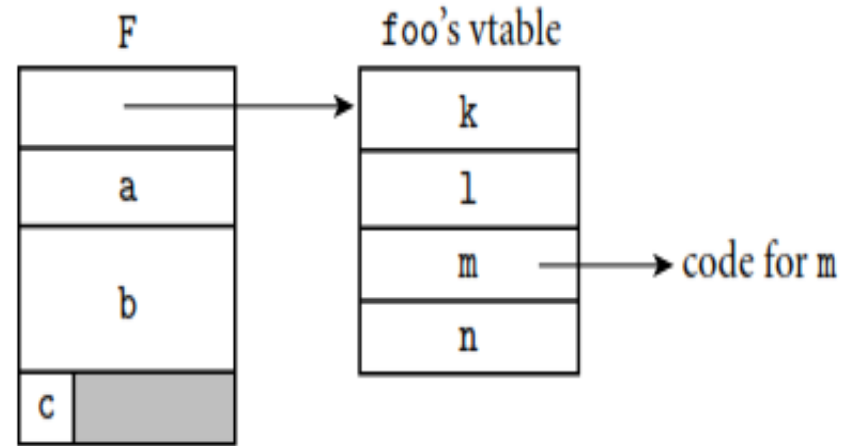


Fig. Implementation of virtual methods. The representation of object `F` begins with the address of the vtable for class `foo`. (All objects of this class will point to the same vtable.) The vtable itself consists of an array of addresses, one for the code of each virtual method of the class. The remainder of `F` consists of the representations of its fields.

```

class bar : public foo {
    int w;
public:
    void m(); //override
    virtual double s( ...
    virtual char *t( ...
    ...
} B;

```

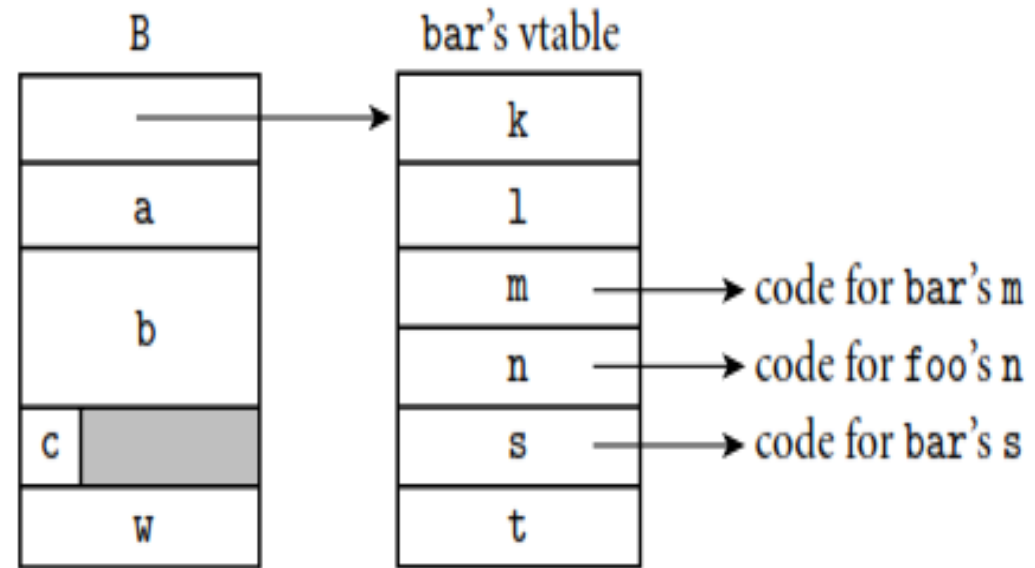


Fig. Implementation of single inheritance. As in [above fig](#) the representation of object B begins with the address of its class's vtable. The first four entries in the table represent the same members as they do for foo, except that one m has been overridden and now contains the address of the code for a different subroutine. Additional fields of bar follow the ones inherited from foo in the representation of B; additional virtual methods follow the ones inherited from foo in the vtable of class bar.

```
class Shape:
```

```
    def name():    print "Shape"
```

```
    def color():   print "Blue"
```

```
class Rectangle(Shape):
```

```
    def name():    print "Rectangle"
```

```
    def color():   print "Red"
```

```
class Square(Shape)
```

```
    def name():    print "Square"
```

```
Square s = new Square()
```

```
Rectangle r = s
```

```
Shape sh = s
```

```
s.name()
```

```
r.name()
```

```
sh.name()
```

Static Method Binding:

Square Rectangle Shape

Dynamic Method Binding:

Square Square Square

Multiple Inheritance

In C++, you can say

```
class professor : public teacher, public researcher {  
  
    ...  
  
}
```

- Here you get all the members of teacher and all the members of researcher
 - If there's anything that's in both (same name and argument types), then calls to the member are ambiguous; the compiler disallows them
- Virtual base classes: In the usual case if you inherit from two classes that are both derived from some other class B, your implementation includes two copies of B's data members

Object-Oriented Programming

- Many OO languages are strictly sequential, but the model adapts well to parallelism as well
- we characterized object-oriented programming in terms of three fundamental concepts:
 - Encapsulation (uniform data abstraction)- everything is an object
 - Inheritance
 - dynamic method binding
- **Smalltalk** is the canonical object-oriented language
 - It has all three of the characteristics listed above
 - It is based on the thesis work of Alan Kay at Utah in the late 1960's
 - It went through 5 generations at Xerox PARC, where Kay worked after graduating
 - Smalltalk-80 is the current standard

Modula-3

- single inheritance
- all methods virtual
- no constructors or destructors

Ada 95

- *tagged types*
- single inheritance
- no constructors or destructors
- *class-wide parameters*:
 - methods static by default
 - can define a parameter or pointer that grabs the object-specific version of all methods
 - o base class doesn't have to decide what will be virtual
- notion of *child packages as an alternative to friends*

Java

- interfaces, *mix-in inheritance*
- alternative to multiple inheritance
 - basically you inherit from one real parent and one or more interfaces, each of which contains *only virtual* functions and no data
 - this avoids the contiguity issues in multiple inheritance above, allowing a very simple implementation
- all methods virtual
- **Smalltalk** is really pretty purely object-oriented
- **C++** can be used in an object-oriented style, but it's arguable that it is a pure object-based design. Why?
 - Note that simple types are not classes
 - Has subroutines outside of classes

End of chapter - 6